🍜 Ramen Inc. — 丼ビジュアル自動生成パイプライン設計書

> Claude Code + Google Gemini API で画像生成→合成を完全自動化する

---

1. 概要

目的

レシピの出汁×タレ×トッピングの全組み合わせに対応する丼ビジュアルを、 スープベース画像（7枚）+ トッピング画像（11枚）のパーツから自動合成する。

パイプライン全体像

```
[Step 1] Gemini APIで画像生成（18枚）
    ↓
[Step 2] 白背景除去（自動）
    ↓
[Step 3] パーツ合成（自動）→ 全組み合わせの丼画像を出力
    ↓
[Step 4] assets/bowl/ に配置 → アプリから参照
```

生成する素材

カテゴリ	枚数	内容
スープベース	7枚	出汁×タレの色違い（トッピングなし）
トッピング	11枚	各具材の単体画像（白背景→透過）
合計生成	**18枚**	Gemini APIで生成
合成結果	必要に応じて	ベース×トッピング組み合わせ

---

2. ディレクトリ構成

```
ramen-inc/
├── scripts/
│   └── generate-bowl-assets/
│       ├── config.ts              # 設定（API Key、パス、プロンプト等）
│       ├── generate-images.ts     # Step 1: Gemini APIで画像生成
│       ├── remove-background.ts   # Step 2: 白背景除去
│       ├── composite.ts           # Step 3: パーツ合成
│       ├── run-all.ts             # 全Step一括実行
│       └── prompts/
│           ├── common-style.txt   # 共通スタイル指示
│           ├── soup-bases.json    # スープベース7枚の定義
│           └── toppings.json      # トッピング11枚の定義
├── assets/
│   └── bowl/
│       ├── raw/                   # Gemini生成の生画像（白背景）
│       │   ├── soup_s1_white_cream.png
│       │   ├── soup_s2_cream_brown.png
│       │   ├── ...
│       │   ├── topping_chashu.png
│       │   ├── topping_ajitama.png
│       │   └── ...
│       ├── parts/                 # 背景除去済みパーツ（透過PNG）
│       │   ├── soup_s1_white_cream.png
│       │   ├── ...
│       │   ├── topping_chashu.png
│       │   └── ...
│       └── composed/             # 合成済み完成品
│           ├── bowl_tori_paitan_shio_chashu_ajitama_nori.png
│           └── ...
```

---

3. 設定ファイル

config.ts

```
export const CONFIG = {
  // Gemini API
  geminiApiKey: process.env.GEMINI_API_KEY,
  geminiModel: "gemini-2.5-flash-image",  // Nano Banana（無料枠）

  // 画像設定
  imageSize: 1024,          // 生成画像のサイズ
  outputSize: 512,          // 最終出力サイズ（アプリ用）
```

```
  // パス
  rawDir: "assets/bowl/raw",
  partsDir: "assets/bowl/parts",
  composedDir: "assets/bowl/composed",
  promptsDir: "scripts/generate-bowl-assets/prompts",

  // 背景除去
  whiteThreshold: 235,      // この値以上のRGBを透過にする

  // 合成設定
  composite: {
    bowlFillRatio: 0.75,    // 丼が画像の75%を占める
    toppingPositions: {
      // 各トッピングの配置位置（丼幅に対する比率）
      chashu:       { x: 0.12, y: 0.32, w: 0.38, h: 0.28 },
      ajitama:      { x: 0.42, y: 0.36, w: 0.36, h: 0.27 },
      nori:         { x: 0.50, y: 0.18, w: 0.30, h: 0.23, rotate: 10, layer: "bac
      menma:        { x: 0.35, y: 0.30, w: 0.20, h: 0.22 },
      negi:         { x: 0.32, y: 0.42, w: 0.18, h: 0.08 },
      shiraga_negi: { x: 0.30, y: 0.38, w: 0.22, h: 0.14 },
      moyashi:      { x: 0.28, y: 0.34, w: 0.24, h: 0.16 },
      butter:       { x: 0.40, y: 0.38, w: 0.14, h: 0.10 },
      corn:         { x: 0.36, y: 0.44, w: 0.16, h: 0.10 },
      yuzu:         { x: 0.44, y: 0.40, w: 0.10, h: 0.07 },
      kikurage:     { x: 0.48, y: 0.34, w: 0.18, h: 0.14 },
    }
  }
};
```

prompts/common-style.txt

```
Style: modern vector illustration. NO black outlines — shapes are defined
by color differences and soft gradients only. Smooth clean shapes like
Adobe Illustrator artwork. Soft inner shadows and highlights give gentle
3D form. Think: premium app icon quality, Figma/Sketch design asset.
NOT anime. NOT cartoon. NOT hand-drawn. NOT cel-shaded. Color fills with
subtle gradients, no flat solid colors. Muted warm palette. Slightly
desaturated, sophisticated. Perspective: top-down 3/4 isometric, looking
into bowl at 45 degrees. Lighting: soft diffused, no harsh shadows.
Bowl: white ceramic with thin red line, smooth rounded form. No chopsticks.
No steam. No background. Image: square, centered, bowl fills 75% of frame.
Outline strokes should be very thin hairline weight using muted warm brown
color, not black or dark brown. No chopsticks.
```

prompts/soup-bases.json

```
[
  {
    "id": "s1_white_cream",
    "name": "白クリーム",
    "covers": ["tori_paitan+shio", "tori_paitan+tonkotsu_tare", "tonkotsu+shio",
    "prompt": "A ramen bowl with thick creamy white broth. The soup is opaque mil
  },
  {
    "id": "s2_cream_brown",
    "name": "クリーム茶",
    "covers": ["tori_paitan+shoyu", "tonkotsu+shoyu"],
    "prompt": "A ramen bowl with creamy brown soy sauce broth. The soup is opaque
  },
  {
    "id": "s3_clear_dark_amber",
    "name": "澄んだ濃い琥珀",
    "covers": ["gyokai+shoyu"],
    "prompt": "A ramen bowl with clear dark amber fish broth. The soup is transpa
  },
  {
    "id": "s4_golden_brown",
    "name": "金茶",
    "covers": ["tori_gyokai+shoyu", "yasai+shoyu", "tori_gyokai+miso"],
    "prompt": "A ramen bowl with golden-brown broth. The soup is a warm rich gold
  },
  {
    "id": "s5_miso_amber",
    "name": "濃い琥珀（味噌）",
    "covers": ["tori_paitan+miso", "tonkotsu+miso", "gyokai+miso", "yasai+miso"],
    "prompt": "A ramen bowl with thick miso broth. The soup is opaque warm amber-
  },
  {
    "id": "s6_clear_light_gold",
    "name": "澄んだ薄金",
    "covers": ["gyokai+shio", "tori_gyokai+shio"],
    "prompt": "A ramen bowl with clear light golden broth. The soup is transparen
  },
  {
    "id": "s7_almost_clear",
    "name": "ほぼ透明（薄黄緑）",
    "covers": ["yasai+shio", "gyokai+tonkotsu_tare", "yasai+tonkotsu_tare", "tori
    "prompt": "A ramen bowl with very light clear vegetable broth. The soup is al
  }
]
```

prompts/toppings.json


```
[
  {
    "id": "chashu",
    "name": "チャーシュー",
    "prompt": "A single round slice of Japanese chashu pork for ramen topping, vi
  },
  {
    "id": "ajitama",
    "name": "味玉",
    "prompt": "A soft-boiled ramen egg ajitama cut in half. The white is light br
  },
  {
    "id": "nori",
    "name": "海苔",
    "prompt": "A single sheet of dried nori seaweed for ramen. Dark green-black r
  },
  {
    "id": "menma",
    "name": "メンマ",
    "prompt": "A small pile of 3-4 menma seasoned bamboo shoots. Pale yellow-brow
  },
  {
    "id": "negi",
    "name": "ネギ",
    "prompt": "A small pile of sliced green onion negi rings, about 8-10 thin rou
  },
  {
    "id": "shiraga_negi",
    "name": "白髪ねぎ",
    "prompt": "A small fluffy mound of shredded white leek shiraga negi. Very thi
  },
  {
    "id": "moyashi",
    "name": "もやし",
    "prompt": "A small handful of bean sprouts moyashi. White thin sprouts with s
  },
  {
    "id": "butter",
    "name": "バター",
    "prompt": "A single small rectangular pat of butter. Bright yellow with a gen
  },
  {
    "id": "corn",
    "name": "コーン",
```

```
    "prompt": "A small scattering of about 15 sweet corn kernels. Bright yellow,
  },
  {
    "id": "yuzu",
    "name": "柚子皮",
    "prompt": "Two or three thin strips of yuzu citrus peel. Bright yellow-green,
  },
  {
    "id": "kikurage",
    "name": "きくらげ",
    "prompt": "Two or three pieces of wood ear mushroom kikurage. Dark brown, thi
  }
]
```

---

4. Step 1: Gemini APIで画像生成

generate-images.ts

```
import { GoogleGenAI } from "@google/genai";
import * as fs from "fs";
import * as path from "path";
import { CONFIG } from "./config";

const client = new GoogleGenAI({ apiKey: CONFIG.geminiApiKey });

// 共通スタイルプロンプトを読み込み
const commonStyle = fs.readFileSync(
  path.join(CONFIG.promptsDir, "common-style.txt"), "utf-8"
);

async function generateImage(promptText: string, outputPath: string) {
  const fullPrompt = `${commonStyle}\n\n${promptText}`;

  const response = await client.models.generateContent({
    model: CONFIG.geminiModel,
    contents: [{ role: "user", parts: [{ text: fullPrompt }] }],
    // Gemini 2.5 Flash Imageは画像生成をネイティブサポート
    generationConfig: {
      responseModalities: ["image", "text"],
    },
  });

  // レスポンスから画像データを抽出
```

```
  for (const part of response.candidates[0].content.parts) {
    if (part.inlineData) {
      const buffer = Buffer.from(part.inlineData.data, "base64");
      fs.writeFileSync(outputPath, buffer);
      console.log(`✅ Generated: ${outputPath}`);
      return;
    }
  }
  console.error(`❌ No image in response for: ${outputPath}`);
}

async function generateAll() {
  // ディレクトリ作成
  fs.mkdirSync(CONFIG.rawDir, { recursive: true });

  // スープベース7枚
  const soupBases = JSON.parse(
    fs.readFileSync(path.join(CONFIG.promptsDir, "soup-bases.json"), "utf-8")
  );
  for (const soup of soupBases) {
    const outputPath = path.join(CONFIG.rawDir, `soup_${soup.id}.png`);
    if (fs.existsSync(outputPath)) {
      console.log(`⏭️ Skip (exists): ${outputPath}`);
      continue;
    }
    await generateImage(soup.prompt, outputPath);
    // レートリミット対策: 2秒待機
    await new Promise(r => setTimeout(r, 2000));
  }

  // トッピング11枚
  const toppings = JSON.parse(
    fs.readFileSync(path.join(CONFIG.promptsDir, "toppings.json"), "utf-8")
  );
  for (const topping of toppings) {
    const outputPath = path.join(CONFIG.rawDir, `topping_${topping.id}.png`);
    if (fs.existsSync(outputPath)) {
      console.log(`⏭️ Skip (exists): ${outputPath}`);
      continue;
    }
    await generateImage(topping.prompt, outputPath);
    await new Promise(r => setTimeout(r, 2000));
  }

  console.log("\n🎉 All images generated!");
}
```

```
generateAll().catch(console.error);
```

API利用料金

```
Gemini 2.5 Flash Image（Nano Banana）：
  無料枠：500 RPD（API経由）
  18枚の生成：無料枠内で余裕

  有料の場合：1枚あたり約$0.04（1024x1024）
  18枚 × $0.04 = $0.72（約¥110）
```

---

5. Step 2: 白背景除去

remove-background.ts

```
// Pythonスクリプトを Claude Codeから実行
// sharp（Node.js）またはPillow（Python）で処理

import { execSync } from "child_process";
import * as fs from "fs";
import * as path from "path";
import { CONFIG } from "./config";

function removeBackground() {
  fs.mkdirSync(CONFIG.partsDir, { recursive: true });

  const rawFiles = fs.readdirSync(CONFIG.rawDir)
    .filter(f => f.endsWith(".png"));

  for (const file of rawFiles) {
    const input = path.join(CONFIG.rawDir, file);
    const output = path.join(CONFIG.partsDir, file);

    if (fs.existsSync(output)) {
      console.log(`⏭️ Skip: ${file}`);
      continue;
    }

    // Pythonで白背景除去
    execSync(`python3 -c "
import numpy as np
from PIL import Image
```

```
img = Image.open('${input}').convert('RGBA')
arr = np.array(img)
threshold = ${CONFIG.whiteThreshold}
white = (arr[:,:,0] > threshold) & (arr[:,:,1] > threshold) & (arr[:,:,2] > thres
arr[white, 3] = 0
Image.fromarray(arr).save('${output}')
print('✅ ${file}')
"`);
  }

  console.log("\n🎉 All backgrounds removed!");
}

removeBackground();
```

---

6. Step 3: パーツ合成

composite.ts

```
import { execSync } from "child_process";
import * as fs from "fs";
import * as path from "path";
import { CONFIG } from "./config";

// スープベースのマッピング（出汁+タレ → スープベースID）
const SOUP_MAP: Record<string, string> = {
  "tori_paitan+shio": "s1_white_cream",
  "tori_paitan+tonkotsu_tare": "s1_white_cream",
  "tonkotsu+shio": "s1_white_cream",
  "tonkotsu+tonkotsu_tare": "s1_white_cream",
  "tori_paitan+shoyu": "s2_cream_brown",
  "tonkotsu+shoyu": "s2_cream_brown",
  "gyokai+shoyu": "s3_clear_dark_amber",
  "tori_gyokai+shoyu": "s4_golden_brown",
  "yasai+shoyu": "s4_golden_brown",
  "tori_gyokai+miso": "s4_golden_brown",
  "tori_paitan+miso": "s5_miso_amber",
  "tonkotsu+miso": "s5_miso_amber",
  "gyokai+miso": "s5_miso_amber",
  "yasai+miso": "s5_miso_amber",
  "gyokai+shio": "s6_clear_light_gold",
  "tori_gyokai+shio": "s6_clear_light_gold",
```

```
  "yasai+shio": "s7_almost_clear",
  "gyokai+tonkotsu_tare": "s7_almost_clear",
  "yasai+tonkotsu_tare": "s7_almost_clear",
  "tori_gyokai+tonkotsu_tare": "s7_almost_clear",
};

// トッピング配置座標（丼幅に対する比率）
// layer: "back" は丼のリムに立てかける系（海苔）→ 他より先に配置
const TOPPING_LAYOUT: Record<string, {
  x: number; y: number; w: number; h: number;
  rotate?: number; layer?: "back" | "front";
}> = {
  chashu:       { x: 0.12, y: 0.32, w: 0.38, h: 0.28 },
  ajitama:      { x: 0.42, y: 0.36, w: 0.36, h: 0.27 },
  nori:         { x: 0.50, y: 0.18, w: 0.30, h: 0.23, rotate: 10, layer: "back" }
  menma:        { x: 0.35, y: 0.28, w: 0.20, h: 0.22 },
  negi:         { x: 0.32, y: 0.44, w: 0.18, h: 0.08 },
  shiraga_negi: { x: 0.30, y: 0.38, w: 0.22, h: 0.14 },
  moyashi:      { x: 0.20, y: 0.34, w: 0.24, h: 0.16 },
  butter:       { x: 0.42, y: 0.40, w: 0.14, h: 0.10 },
  corn:         { x: 0.36, y: 0.44, w: 0.16, h: 0.10 },
  yuzu:         { x: 0.44, y: 0.42, w: 0.10, h: 0.07 },
  kikurage:     { x: 0.48, y: 0.34, w: 0.18, h: 0.14 },
};

interface CompositeRequest {
  broth: string;       // 出汁ID
  tare: string;        // タレID
  toppings: string[];  // トッピングIDの配列
  outputName: string;  // 出力ファイル名
}

function composeBowl(request: CompositeRequest) {
  const soupKey = `${request.broth}+${request.tare}`;
  const soupId = SOUP_MAP[soupKey];
  if (!soupId) {
    console.error(`❌ Unknown soup combo: ${soupKey}`);
    return;
  }

  const soupPath = path.join(CONFIG.partsDir, `soup_${soupId}.png`);
  const outputPath = path.join(CONFIG.composedDir, `${request.outputName}.png`);

  // トッピングをback/frontレイヤーに分離
  const backToppings = request.toppings.filter(t => TOPPING_LAYOUT[t]?.layer ===
  const frontToppings = request.toppings.filter(t => TOPPING_LAYOUT[t]?.layer !==
  const orderedToppings = [...backToppings, ...frontToppings];
```

```
  // トッピングの配置情報をJSON化してPythonに渡す
  const toppingArgs = orderedToppings.map(id => {
    const layout = TOPPING_LAYOUT[id];
    const toppingPath = path.join(CONFIG.partsDir, `topping_${id}.png`);
    return JSON.stringify({ id, path: toppingPath, ...layout });
  });

  const pythonScript = `
import json, sys
from PIL import Image
import numpy as np

# Load bowl base
bowl = Image.open("${soupPath}").convert("RGBA")
bw = bowl.width
result = bowl.copy()

# Toppings
toppings = ${JSON.stringify(orderedToppings.map(id => ({
    id,
    path: path.join(CONFIG.partsDir, `topping_${id}.png`),
    ...TOPPING_LAYOUT[id]
  }))))}

for tp in toppings:
    try:
        img = Image.open(tp["path"]).convert("RGBA")
        tw = int(bw * tp["w"])
        th = int(bw * tp["h"])
        img_r = img.resize((tw, th), Image.LANCZOS)
        if tp.get("rotate"):
            img_r = img_r.rotate(tp["rotate"], expand=True, resample=Image.BICUBI
        px = int(bw * tp["x"])
        py = int(bw * tp["y"])
        result.paste(img_r, (px, py), img_r)
        print(f"  ✅ {tp['id']}")
    except Exception as e:
        print(f"  ❌ {tp['id']}: {e}")

# Resize to output size
result = result.resize((${CONFIG.outputSize}, ${CONFIG.outputSize}), Image.LANCZO
result.save("${outputPath}")
print(f"✅ Saved: ${outputPath}")
`;

  execSync(`python3 -c '${pythonScript.replace(/'/g, "\\'")}'`);
```

```
}

// ゲーム内で使用するプリセット合成パターン
function composePresets() {
  fs.mkdirSync(CONFIG.composedDir, { recursive: true });

  const presets: CompositeRequest[] = [
    // 代表的な組み合わせ
    { broth: "tori_paitan", tare: "shio", toppings: ["nori", "chashu", "ajitama",
    { broth: "tonkotsu", tare: "tonkotsu_tare", toppings: ["chashu", "negi", "nor
    { broth: "gyokai", tare: "shoyu", toppings: ["nori", "chashu", "ajitama", "me
    { broth: "yasai", tare: "miso", toppings: ["chashu", "corn", "butter", "moyas
    // ... 必要に応じて追加
  ];

  for (const preset of presets) {
    console.log(`\n🍜 Composing: ${preset.outputName}`);
    composeBowl(preset);
  }
}

composePresets();
```

---

7. アプリ側（React Native）での動的合成

方式A: プリセット画像方式（推奨・MVP）

事前に代表的な組み合わせを合成しておき、最も近いプリセットを表示。

```
// bowlImageMap.ts
const SOUP_MAP: Record<string, string> = {
  "tori_paitan+shio": "s1_white_cream",
  "tori_paitan+tonkotsu_tare": "s1_white_cream",
  // ... 全20組み合わせ
};

export function getBowlImage(broth: string, tare: string): ImageSourcePropType {
  const key = `${broth}+${tare}`;
  const soupId = SOUP_MAP[key] || "s1_white_cream";

  // スープベースごとに代表的な合成画像を1枚ずつ用意
  const images: Record<string, ImageSourcePropType> = {
    s1_white_cream: require("../assets/bowl/composed/s1_default.png"),
    s2_cream_brown: require("../assets/bowl/composed/s2_default.png"),
```

```
    s3_clear_dark_amber: require("../assets/bowl/composed/s3_default.png"),
    s4_golden_brown: require("../assets/bowl/composed/s4_default.png"),
    s5_miso_amber: require("../assets/bowl/composed/s5_default.png"),
    s6_clear_light_gold: require("../assets/bowl/composed/s6_default.png"),
    s7_almost_clear: require("../assets/bowl/composed/s7_default.png"),
  };

  return images[soupId];
}
```

方式B: ランタイム合成方式（v1.1以降）

React Native上でリアルタイムにパーツを重ねて表示。

```
// BowlVisual.tsx
import { View, Image, StyleSheet } from "react-native";

interface BowlVisualProps {
  broth: string;
  tare: string;
  toppings: string[];
}

const soupImages = {
  s1_white_cream: require("../assets/bowl/parts/soup_s1_white_cream.png"),
  // ...
};

const toppingImages = {
  chashu: require("../assets/bowl/parts/topping_chashu.png"),
  ajitama: require("../assets/bowl/parts/topping_ajitama.png"),
  // ...
};

const TOPPING_STYLES: Record<string, object> = {
  chashu: { left: "12%", top: "32%", width: "38%", height: "28%" },
  ajitama: { left: "42%", top: "36%", width: "36%", height: "27%" },
  nori: { left: "50%", top: "18%", width: "30%", height: "23%", zIndex: 1 },
  // ... 全11種
};

export function BowlVisual({ broth, tare, toppings }: BowlVisualProps) {
  const soupId = getSoupId(broth, tare);

  // backレイヤー（海苔など）を先に、frontを後に
  const backToppings = toppings.filter(t => TOPPING_STYLES[t]?.zIndex === 1);
```

```
  const frontToppings = toppings.filter(t => TOPPING_STYLES[t]?.zIndex !== 1);

  return (
    <View style={styles.container}>
      {/* スープベース */}
      <Image source={soupImages[soupId]} style={styles.base} />

      {/* Back layer toppings */}
      {backToppings.map(id => (
        <Image key={id} source={toppingImages[id]}
          style={[styles.topping, TOPPING_STYLES[id]]} />
      ))}

      {/* Front layer toppings */}
      {frontToppings.map(id => (
        <Image key={id} source={toppingImages[id]}
          style={[styles.topping, TOPPING_STYLES[id]]} />
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  container: { width: 200, height: 200, position: "relative" },
  base: { width: "100%", height: "100%", position: "absolute" },
  topping: { position: "absolute", resizeMode: "contain" },
});
```

---

8. 実行手順

前提条件

```
# Google AI Studio でAPIキーを取得（無料）
# https://aistudio.google.com/apikey
export GEMINI_API_KEY="your-api-key-here"

# 依存パッケージ
pip install Pillow numpy --break-system-packages
npm install @google/genai
```

一括実行

```
# 全ステップを順番に実行
npx ts-node scripts/generate-bowl-assets/run-all.ts

# または個別に実行
npx ts-node scripts/generate-bowl-assets/generate-images.ts  # Step 1
npx ts-node scripts/generate-bowl-assets/remove-background.ts # Step 2
npx ts-node scripts/generate-bowl-assets/composite.ts          # Step 3
```

所要時間・コスト

```
Step 1（画像生成）: 18枚 × 約5秒 = 約2分（API待機含めて約5分）
Step 2（背景除去）: 18枚 × 約0.5秒 = 約10秒
Step 3（合成）:     パターン数 × 約1秒

合計: 約5〜10分
コスト: 無料（Gemini API無料枠内）
```

---

9. 品質チェック

自動チェック項目

- □ 全18枚のraw画像が生成されているか
- □ 全18枚のparts画像（透過）が生成されているか
- □ スープベース7枚の色が明確に異なるか
- □ トッピング11枚の白背景が完全に除去されているか
- □ 合成画像でトッピングが丼の外にはみ出していないか
- □ 合成画像でトッピング同士が不自然に重なっていないか

手動チェック（生成後に目視確認）

- □ スタイルが全画像で統一されているか
- □ 丼のアングル・サイズが全画像で一致しているか
- □ スープの色がプロンプト通りか
- □ トッピングのサイズ感が自然か
- □ 全体として「美味しそう」に見えるか

リトライ機能

Gemini APIの出力はnon-deterministicなので、品質が低い画像は再生成できる。 `generate-`

`images.ts` は既存ファイルをスキップするので、 不満な画像を `raw/` から削除して再実行すればその画像だけ再生成される。

---

10. 将来の拡張

v1.1: ランタイム合成

- React Native上でパーツを動的に重ねる（方式B）
- プレイヤーのレシピに完全に対応した丼ビジュアル

v1.2: 追加トッピング

- toppings.jsonに追加するだけで新トッピングが使える
- 合成スクリプトは変更不要

v2: 都市DLC

- 丼のデザイン（器の色・模様）を都市ごとに変更
- スープベースの器部分だけ差し替える仕組み